



Cloud Management Mechanisms

Resource, Architecture, Security & Governance

RAVI SAJJANAR

Senior Cloud Developer
Hewlett Packard Enterprise

Cloud Management Mechanisms

Session 1:

1. Cloud Computing Fundamentals
2. Cloud Computing Architectures
 - Workload Distribution
 - Elastic Resource Capacity
 - Multi Cloud
 - Hypervisor Clustering
 - Cloud Balancing
3. Specialized Cloud Architectures:
 - Edge Computing
 - Fog Computing
 - Meta Cloud
 - Federated Cloud

Session 2:

1. Cloud Computing Security
 - Threat Agents
 - Common Threats
2. Access-Oriented Mechanisms
 - Security Groups
 - Hardened Image
 - IAM System
3. Data-Oriented Mechanisms:
 - Data Loss Prevention (DLP) System
 - Trusted Platform Module (TPM)

Agenda

#	Topic	Keywords
1	Security Basics	Threats, Agents
2	Access Control	SG, IAM, Hardened AMI
3	Data Protection	DLP, TPM, Encryption
4	Shared Responsibility	Provider vs Customer

2.1 Cloud Security Fundamentals

Why Security Matters: The Numbers

SECURITY BREACH COSTS (2025)



- Average data breach cost: **\$4.88 MILLION**
- Healthcare breach: **\$11.5 MILLION**
- Capital One (2019): **\$190 MILLION fine**
- Equifax (2017): **\$700 MILLION settlement**



- Average time to detect breach: **194 DAYS**
- Average time to contain: **64 DAYS**



"One wrong checkbox in AWS console = millions in damages"

2.1 Cloud Security Fundamentals

Shared Responsibility Model

AWS Responsible For	You Responsible For
Physical Security	IAM & Access Control
Network Infrastructure	Security Groups
Data Center Operations	Data Encryption
Hypervisor & Hardware	OS/Application Patching
Compliance of Cloud	Security in the Cloud

2.1 Cloud Security Fundamentals

Shared Responsibility Model - Rented Apartment Analogy

Apartment Analogy	AWS Cloud Equivalent	Responsibility
Building structure	Physical data centers	AWS
Main gate security	Network infrastructure	AWS
Electricity & water supply	Compute, storage, hypervisor	AWS
Fire safety systems	Compliance certifications	AWS
Common area maintenance	Managed service patching	AWS
Locking your apartment door	Security Groups, IAM	You
Deciding who gets keys	User access management	You
Protecting valuables inside	Data encryption	You
Closing windows before leaving	EC2 OS patching & updates	You
What you keep in the apartment	Applications & data	You
Organizing belongings safely	Data backup & security policies	You

2.1 Cloud Security Fundamentals

Key Lesson: **AWS secures the cloud. You secure your data and applications in the cloud.** 🌥️🔒

Scenario	Whose Fault?
Building entrance security fails	AWS
Data center power outage	AWS
Your Security Group allows 0.0.0.0/0	You
Weak passwords or leaked credentials	You
Unpatched EC2 instance gets hacked	You
Unencrypted sensitive data is exposed	You

2.1 Cloud Security Fundamentals

Responsibility Changes by Service Type

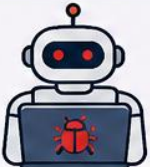
Service Type	Customer Manages	AWS Manages
IaaS (EC2)	OS, patches, applications, data, Security Groups	Hardware, storage, networking, hypervisor
PaaS (Elastic Beanstalk)	Application code, application configuration, data	OS, runtime, middleware, scaling, infrastructure
SaaS (WorkMail, Amazon Connect, etc.)	Data, users, permissions	Application, platform, infrastructure, security, updates

Model	You Manage
IaaS	Most things
PaaS	Just your code and data
SaaS	Almost nothing except users and data

Rule: As you move from **IaaS** → **PaaS** → **SaaS**, your responsibility decreases and the cloud provider's responsibility increases. 🧠 📈

2.1 Cloud Security Fundamentals

The 5 Types of Threat Agents (WHO Attacks?)

1		EXTERNAL HACKERS  • Cybercriminals (want money) • Nation-states (espionage) • Hacktivists (ideology)	 EXAMPLE: SolarWinds Attack (2020) - Russian hackers compromised 18,000+ organizations
2		MALICIOUS INSIDERS  (MOST DANGEROUS!) • Current/former employees with access • Contractors and vendors	 EXAMPLE: Tesla (2020) - Employee offered \$1M to install malware
3		ANONYMOUS ATTACKERS  • Use Tor/VPN to hide identity • Ransomware operators • DDoS attackers	 EXAMPLE:
4		TRUSTED THIRD-PARTY  • Vendors with legitimate access • Supply chain attacks	 EXAMPLE: Target (2013) - Hackers entered via HVAC vendor → 40 million credit cards stolen
5		AUTOMATED THREATS (BOTS)  • Malware/Ransomware • 24/7 scanning for exposed credentials • Crypto miners	 EXAMPLE: Mirai Botnet (2016) - 600K infected IoT devices took down Twitter, Netflix, Reddit
 KEY INSIGHT:  Insiders are MOST dangerous (already have access)  Bots are MOST common (automated 24/7)			

2.1 Cloud Security Fundamentals



AVOID THESE MISTAKES!



#1: Using root account for daily work

→ Root = God mode.
One mistake = total disaster



#2: No MFA (Multi-Factor Authentication)

→ Password alone can be stolen.
MFA = much harder.



#3: S3 bucket set to "public"

→ Your data visible to
ENTIRE INTERNET!



#4: Security Group allows SSH (port 22) from 0.0.0.0/0

→ Anyone can try to
hack your server



#5: Giving "AdministratorAccess" to everyone

→ Intern can delete
production database!



#6: Hardcoding passwords/keys in code

→ Gets pushed to GitHub,
hackers find it in minutes



80% of breaches are caused by **these 6 mistakes!**

2.1 Cloud Security Fundamentals

Case Study: Capital One Breach (2019)

What Happened?

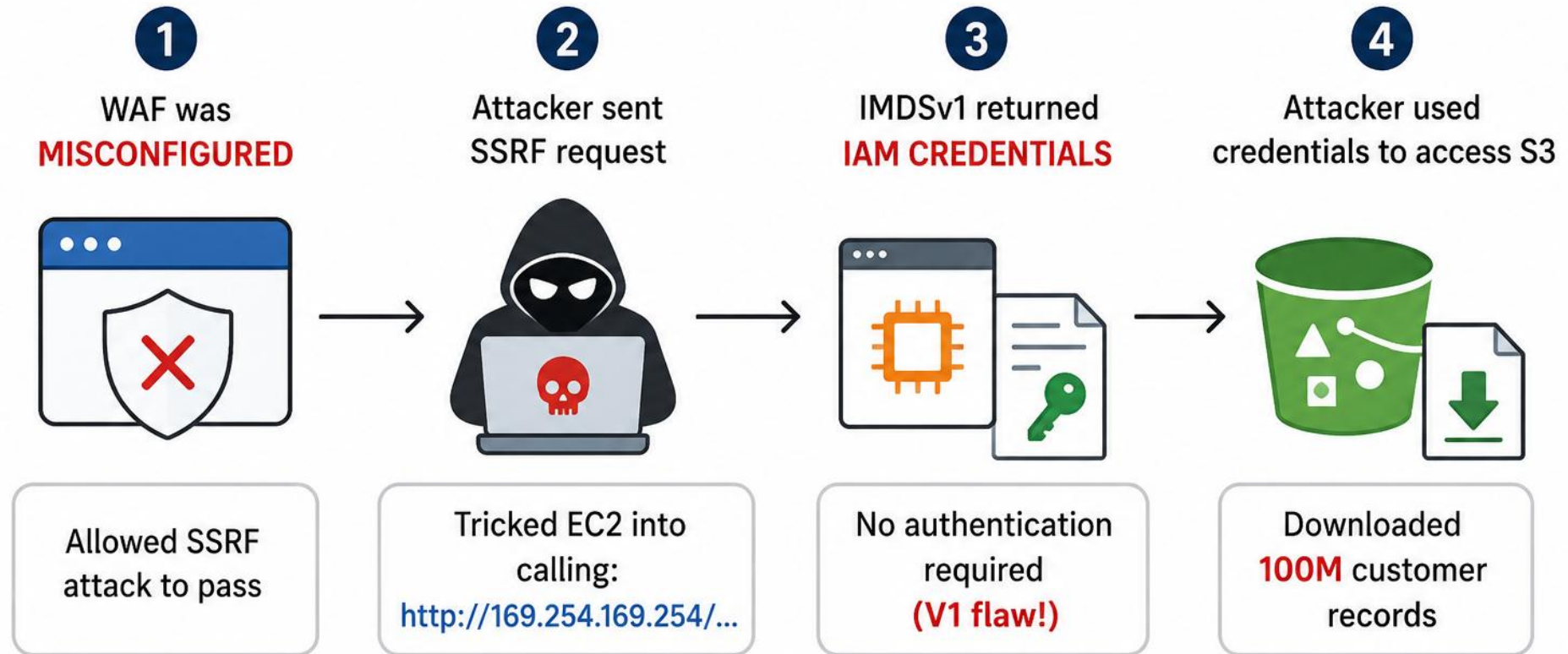
- 1.The attacker exploited a **misconfigured Web Application Firewall (WAF)**.
 - A security setting is configured incorrectly, leaving a door open that should have been closed.
 - It was configured in a way that allowed an **SSRF (Server-Side Request Forgery)** attack.
- 2.Using an **SSRF (Server-Side Request Forgery)** attack, the attacker accessed the EC2 **Instance Metadata Service (IMDSv1)**.
- 3.The attacker obtained temporary **IAM role credentials** assigned to the EC2 instance.
- 4.Using those credentials, the attacker accessed Amazon S3 and downloaded approximately **100 million customer records**.

Note: IMDSv2 Fix: Requires a token, so even if SSRF tricks your server, attacker can't get credentials.

Term	Full Name	Simple Explanation
WAF	Web Application Firewall	A filter that sits in front of your web app, blocking malicious requests (SQL injection, XSS attacks). Like a bouncer checking IDs before letting people into a club.
SSRF	Server-Side Request Forgery	Attack where attacker tricks YOUR server into making requests to internal resources. Like calling a company's internal phone from outside by asking receptionist to dial for you.
IMDSv1	Instance Metadata Service v1	A URL (<code>http://169.254.169.254</code>) every EC2 can access to get info about itself (instance ID, IAM credentials). V1 had no authentication - anyone who could make the server request it got the data.

2.1 Cloud Security Fundamentals

Case Study: Capital One Breach (2019)



FIX: Use **IMDSv2** (requires session token – SSRF can't get it)

2.1 Cloud Security Fundamentals

Case Study: Capital One Breach (2019)

Root Causes

Issue	Impact
Excessive IAM permissions	EC2 role had access to more S3 data than necessary
IMDSv1 enabled	Allowed metadata access without session tokens
Lack of anomaly detection	Large-scale data download went unnoticed
Misconfigured WAF	Created the initial entry point for the attacker

Key Lessons Learned:

- ✓ Use **IMDSv2** instead of IMDSv1 (requires session tokens)
- ✓ Follow the **Principle of Least Privilege** for IAM roles
- ✓ Enable **Amazon GuardDuty** for threat detection
- ✓ Monitor **AWS CloudTrail** logs for unusual S3 activity
- ✓ Regularly review IAM permissions and S3 access patterns

2.1 Cloud Security Fundamentals

The Big 7 Cloud Security Threats

#	Threat	Real-World Example	Prevention
1	Data Breaches	Capital One (2019) - 100M records via misconfigured WAF	Least privilege, encryption, Macie
2	Account Hijacking	Twitter Bitcoin Scam (2020) - Obama, Musk accounts hijacked via social engineering	MFA, employee training
3	Insecure APIs	Parler (2021) - 70TB data scraped via insecure API	Authentication, rate limiting
4	DDoS Attacks	AWS (2020) - 2.3 Tbps attack mitigated	AWS Shield, WAF, auto-scaling
5	Malware Injection	Codecov (2021) - CI/CD tool compromised, stole secrets	Pin dependencies, verify checksums
6	Data Loss	GitLab (2017) - Engineer deleted prod DB, backups didn't work!	Test backups, deletion protection
7	Shared Tech Vulnerabilities	Meltdown/Spectre (2018) - CPU flaws affected all clouds	Keep patched, use dedicated hosts

Key Takeaway: "Most breaches aren't sophisticated hacks - they're misconfigurations, weak passwords, and human error. The good news? These are PREVENTABLE!"

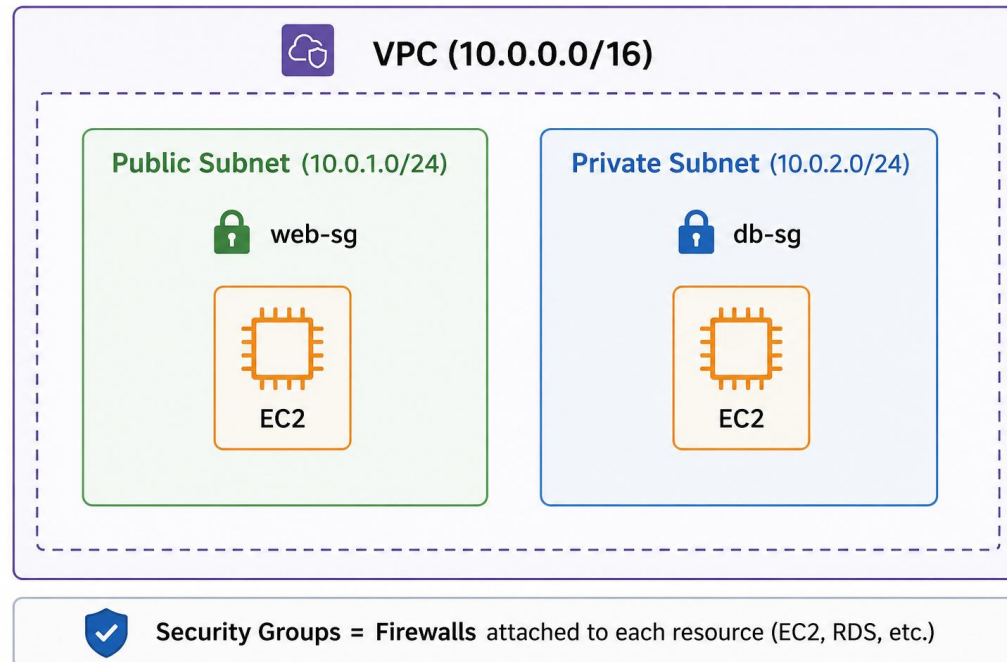
2.2 Security Groups (Virtual Firewalls)

Quick VPC Primer (Before We Start)

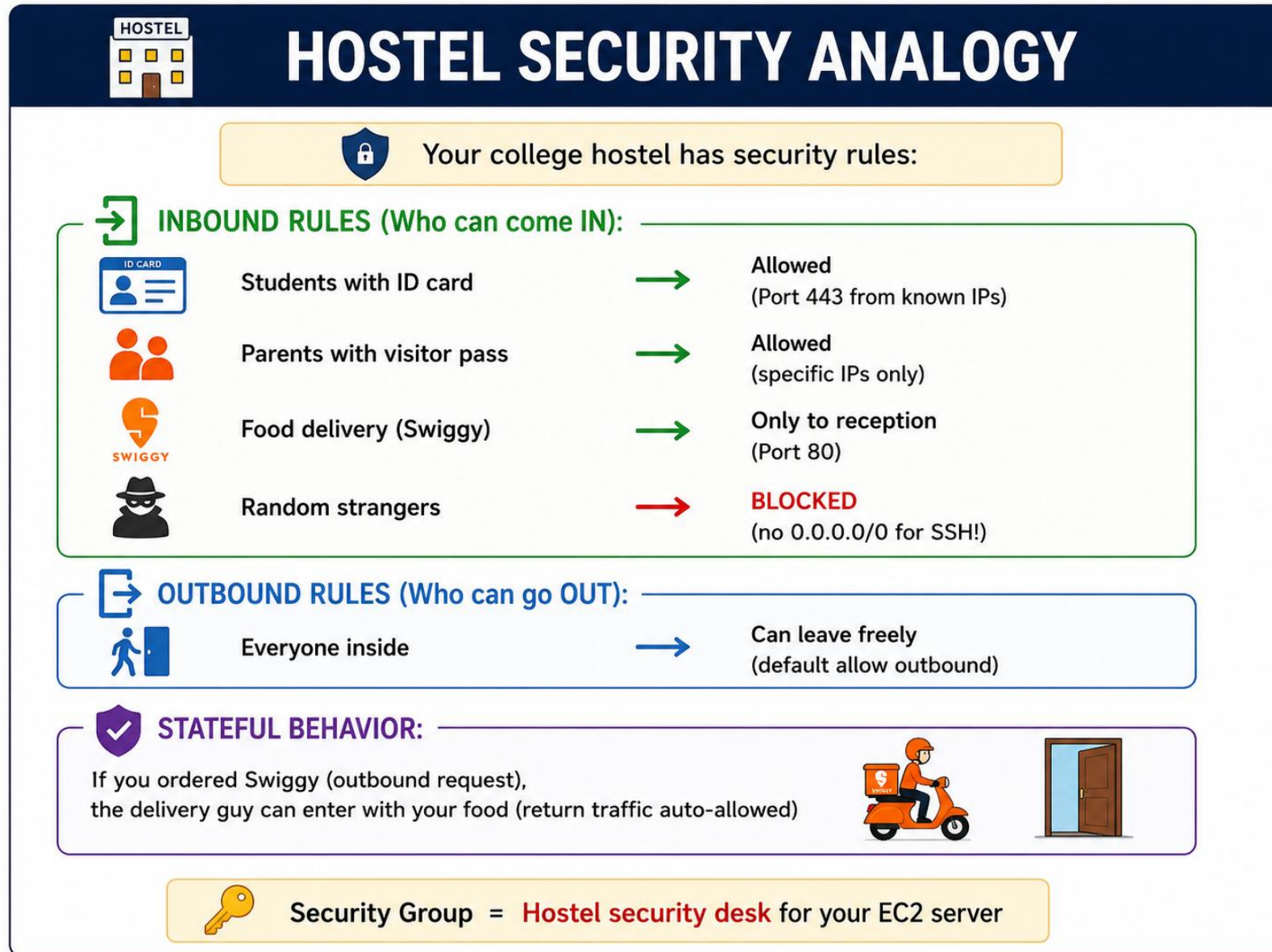
What is a VPC? A Virtual Private Cloud is your own isolated network in AWS - like having your own private data center. Security Groups live inside VPCs and control traffic to your resources.

Security Group Concept

Stateful firewalls controlling inbound/outbound traffic at the instance level. Default behavior: ALL inbound traffic is DENIED, ALL outbound is allowed.



2.2 Security Groups (Virtual Firewalls)



2.2 Security Groups (Virtual Firewalls)

Application Examples

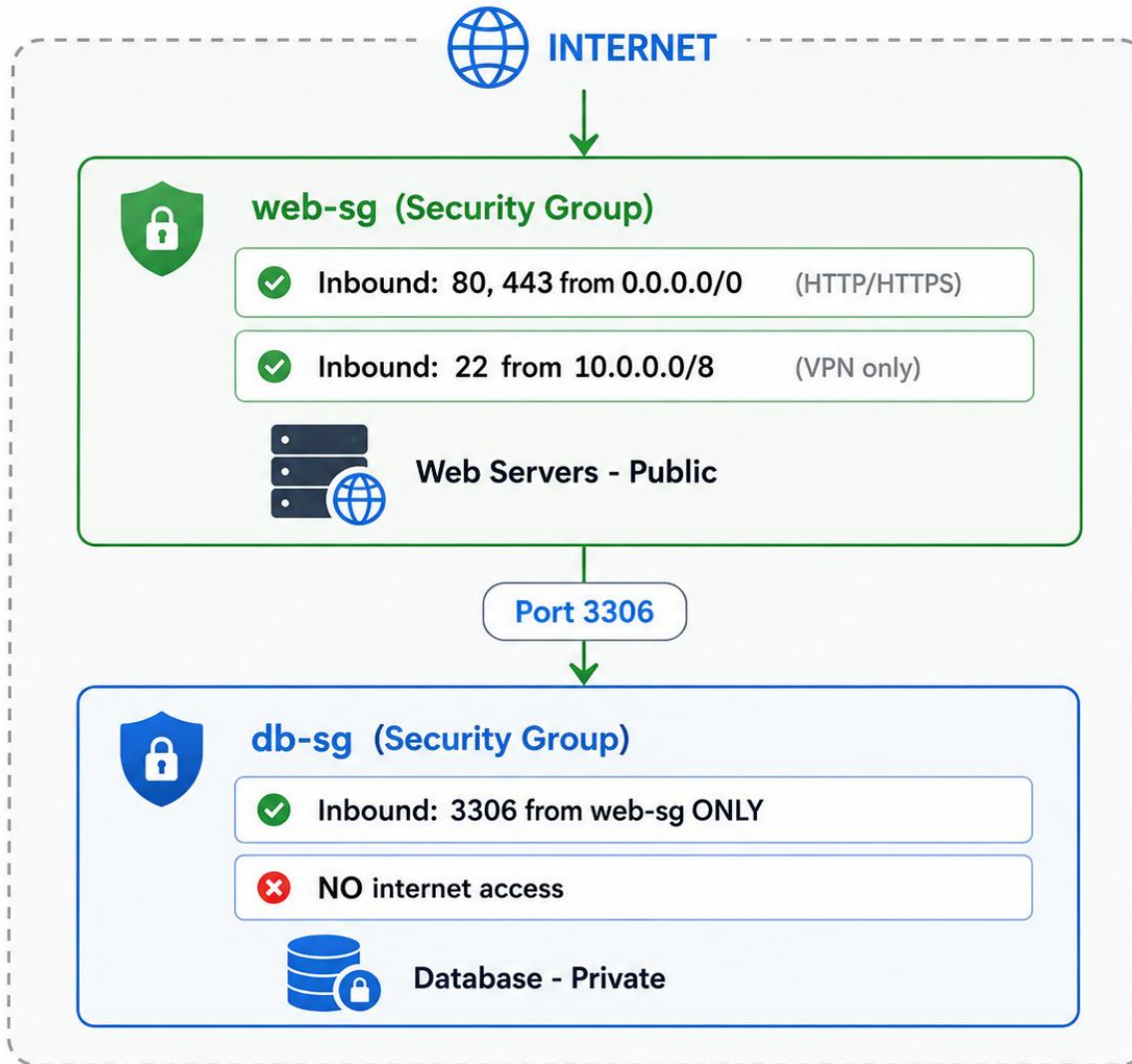
Scenario	Security Group Rule	Real-World Example
Public website	Allow 80/443 from anywhere (0.0.0.0/0)	Flipkart homepage - anyone can browse
Admin panel	Allow 443 only from office IP (203.0.113.0/24)	Flipkart seller dashboard - only registered sellers
Database	Allow 3306 only from app-server security group	Customer data - only Flipkart app can access
SSH access	Allow 22 only from VPN (never 0.0.0.0/0!)	Server maintenance - only IT team via secure tunnel
Game server	Allow UDP 7777-7778 from players	PUBG/Valorant - game traffic only

Key Properties

- **Default:** All inbound denied, all outbound allowed
- **Stateful:** Return traffic automatically allowed
- **Instance-level:** Applied to ENI (Elastic Network Interface)
- **Allow rules only:** Cannot create deny rules

2.2 Security Groups (Virtual Firewalls)

Best Practices Architecture



2.2 Security Groups (Virtual Firewalls)



WOW Moment: Open Port = Instant Attack



Open port **3389** (RDP) to the world attacked within **15 MINUTES**



600+ hack attempts **PER DAY** per server with open ports



IPs scan **EVERY IP** address 24/7 looking for open ports



No firewall = your server IP is on the internet for all to see

2.2 Security Groups (Virtual Firewalls)

Hardened Server Images (AMIs)

Pre-secured AMI templates with security configurations baked in.

Where to Get Hardened AMIs:

- AWS Marketplace - Search "CIS hardened" (some free, some paid)
- Create your own: Launch → Harden → Create AMI → Mandate for all servers
- AWS Systems Manager Patch Manager for automatic updates

2.2 Security Groups (Virtual Firewalls)

STANDARD vs HARDENED SERVER IMAGE

DEFAULT AMAZON LINUX (Convenient but risky!)	HARDENED IMAGE (CIS Benchmark) (Secure by default)
 SSH allows root login	 Root login disabled
 Password auth enabled	 SSH key-only auth
 Many services running	 Only essential services
 No firewall rules	 Firewall configured
 No auto-updates	 Auto security updates
 Default passwords	 CIS Benchmark compliance
 No audit logging	 Full audit logging
 Setup time: 2–4 hours/server	 Setup time: 0 hours (pre-done!)

2.2 Security Groups (Virtual Firewalls)

CLI Configuration

```
# Create web security group
aws ec2 create-security-group \
  --group-name web-sg \
  --description "Web server security group" \
  --vpc-id vpc-12345

# Allow HTTP/HTTPS from anywhere
aws ec2 authorize-security-group-ingress \
  --group-id sg-web123 \
  --protocol tcp \
  --port 80 \
  --cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress \
  --group-id sg-web123 \
  --protocol tcp \
  --port 443 \
  --cidr 0.0.0.0/0

# Create database security group
aws ec2 create-security-group \
  --group-name db-sg \
  --description "Database security group"

# Allow MySQL only from web security group (NOT IP!)
aws ec2 authorize-security-group-ingress \
  --group-id sg-db456 \
  --protocol tcp \
  --port 3306 \
  --source-group sg-web123
```

2.2 Security Groups (Virtual Firewalls)

NACL (Network Access Control List)

Aspect	Details
What it is	A firewall at the subnet level (not instance level)
Where it sits	Between internet/other subnets and your subnet
Default behavior	Allow ALL traffic (unlike Security Groups which deny all inbound)

Security Groups vs NACL - Simple Analogy

NACL = Main Gate Security (Hostel Building Entrance)

- └ Checks EVERYONE entering the building
- └ Rules apply to ALL rooms on that floor (subnet)
- └ Can ALLOW or DENY (turn people away at gate)
- └ Stateless: Checks both entry AND exit separately

Security Group = Room Door Lock (Individual Room)

- └ Checks only people coming to YOUR room
- └ Rules apply to ONE room (instance) only
- └ Can only ALLOW (no explicit deny)
- └ Stateful: If you let someone in, they can leave freely

2.2 Security Groups (Virtual Firewalls)

In most cases: Security Groups are enough. Use NACL for extra defense layer or when you need explicit DENY rules.

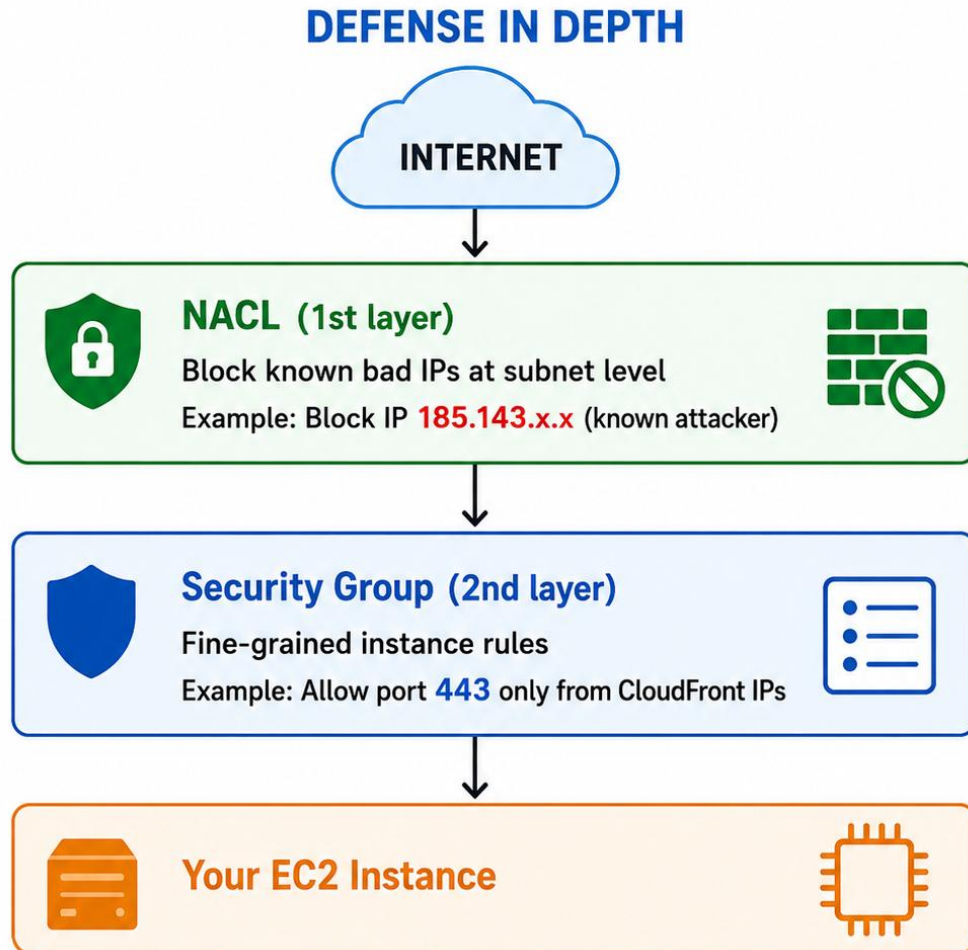
```
# Block a malicious IP at subnet level
aws ec2 create-network-acl-entry \
  --network-acl-id acl-xxx \
  --rule-number 50 \
  --protocol -1 \
  --rule-action deny \
  --cidr-block 185.143.220.0/24 \
  --ingress
```

When to Use NACL

Use Case	Why NACL
Block specific IPs	NACL can DENY (Security Groups can't)
Subnet-wide rules	One rule protects ALL instances in subnet
Compliance	Some regulations require network-level controls
Emergency block	Quickly block attacking IP across all instances

2.2 Security Groups (Virtual Firewalls)

Why Use NACL When We Have Security Groups?



Key Differences

Feature	Security Group	NACL
Level	Instance (EC2, RDS)	Subnet
State	Stateful (return traffic auto-allowed)	Stateless (must allow both directions)
Rules	Allow only	Allow AND Deny
Evaluation	All rules checked	Rules checked in order (first match wins)
Default	Deny all inbound	Allow all



TWO LOCKS ARE BETTER THAN ONE!

NACL = Subnet level (broad rules)

Security Group = Instance level (fine-grained rules)



NACL = Stateless (allow + deny rules)



Security Group = Stateful (allow rules only)

2.3 IAM (Identity and Access Management)

IAM stands for **Identity and Access Management**. It is a framework of policies, processes, and technologies that ensures the right individuals access the exact IT resources they need—at the right time, for the right reasons, and from secure locations



IAM (Identity and Access Management)

Controls WHO can access AWS and WHAT they can do



Users / Groups /
Roles



Permissions
Policies



MFA (Multi-Factor
Authentication)



Least Privilege
Access

Example:

- ✓ Allow developer to start EC2
- ✓ Allow admin full access
- ✓ Deny deleting S3 bucket

2.3 IAM (Identity and Access Management)

IAM helps you securely control access to AWS resources.

THE THREE QUESTIONS IAM ANSWERS:



1 WHO are you? →



IDENTITY (Users, Roles)

IAM identifies who (or what) is making the request.



2 PROVE it! →



AUTHENTICATION (Password, MFA)

IAM verifies the identity using credentials like password or MFA.



3 WHAT can you do? →



AUTHORIZATION (Policies)

IAM determines what actions the identity is allowed to perform.



In short: IAM ensures the **right people** have the **right access** for the **right reasons**.

2.3 IAM (Identity and Access Management)

Understanding IAM Policies

```
{
  "Version": "2012-10-17",           // Policy language version
  "Statement": [                     // List of permissions (one or more statements)
    {
      "Effect": "Allow",              // Allow or Deny the actions
      "Action": [                    // What actions are allowed or denied
        "s3:GetObject",              // Read files (get object from S3)
        "s3:PutObject"              // Write files (upload object to S3)
      ],
      "Resource": "arn:aws:s3::company-data/*", // To which resources
                                              // This bucket and all objects inside it
      "Condition": {                 // Under what conditions the access is allowed
        "IpAddress": {              // Restrict access by IP address
          "aws:SourceIp": "203.0.113.0/24" // Office IP range only
        },
        "Bool": {                    // Additional conditions (true/false)
          "aws:MultiFactorAuthPresent": "true" // MFA is required
        }
      }
    }
  ]
}
```

Navigate: IAM → Users → Create User

	Version	: Policy language version
	Effect	: Allow or Deny the actions
	Action	: What actions are allowed or denied
	Resource	: To which AWS resources the policy applies
	Condition	: Under what conditions the access is allowed

Translation:

"You can read/write files in 'company-data' bucket, but ONLY if you're in the office AND have MFA enabled"

2.3 IAM (Identity and Access Management)

IAM Best Practices

1. NEVER USE ROOT ACCOUNT
 - └─ Root = God mode. Create IAM admin user instead.
 - └─ Enable MFA on root immediately!
2. ENABLE MFA FOR ALL USERS
 - └─ Password can be stolen. MFA = phone is needed too.
 - └─ Makes account hijacking 99% harder.
3. LEAST PRIVILEGE
 - └─ Give minimum permissions needed to do the job.
 - └─ Developer needs EC2? Don't give "AdministratorAccess"!
 - └─ Start with NONE, add as needed.
4. USE GROUPS, NOT INDIVIDUAL PERMISSIONS
 - └─ Create groups: "Developers", "Admins", "Finance"
 - └─ Add users to groups. Much easier to manage!
5. USE ROLES FOR SERVICES
 - └─ EC2 needs S3 access? Give it a ROLE, not keys.
 - └─ Keys can leak. Roles are automatic and secure.
6. ROTATE CREDENTIALS REGULARLY
 - └─ Access keys should be rotated every 90 days.
 - └─ AWS Config can enforce this.
7. REMOVE UNUSED USERS/PERMISSIONS
 - └─ Employee leaves? Delete account SAME DAY.
 - └─ Review access quarterly.

2.3 IAM (Identity and Access Management)

Core Components

Component	Purpose	Example
User	Individual identity	john@company.com
Group	Collection of users	Developers, Admins
Role	Assumed by services/users	EC2-S3-Role
Policy	JSON permission document	S3ReadOnlyAccess

IAM Best Practices

1. **Never use root account** - Create IAM admin user
2. **Enable MFA** - On all human users, especially privileged
3. **Least privilege** - Start with no permissions, add as needed
4. **Use groups** - Don't attach policies directly to users
5. **Use roles for services** - Never embed credentials in code
6. **Rotate credentials** - Access keys every 90 days
7. **Remove unused users** - Audit quarterly

2.3 IAM (Identity and Access Management)

CLI Commands

Create user

```
aws iam create-user --user-name developer1
```

Create group and add user

```
aws iam create-group --group-name Developers
```

```
aws iam add-user-to-group --user-name developer1 --group-name Developers
```

Attach managed policy to group

```
aws iam attach-group-policy \  
    --group-name Developers \  
    --policy-arn arn:aws:iam::aws:policy/AmazonEC2ReadOnlyAccess
```

Create role for EC2

```
aws iam create-role \  
    --role-name EC2-S3-Access \  
    --assume-role-policy-document file://trust-policy.json
```

Enable MFA

```
aws iam enable-mfa-device \  
    --user-name developer1 \  
    --serial-number arn:aws:iam::123456789:mfa/developer1 \  
    --authentication-code1 123456 \  
    --authentication-code2 789012
```

2.4 Data Security Mechanisms

TOPIC A: Data Loss Prevention (DLP)

Core Concept

DLP scans for sensitive patterns (API keys, SSNs, credit cards) and blocks them from being emailed, uploaded, or pushed to GitHub. Like .gitignore for your entire organization.



WOW Moment: Data Breach Reality

 Twitch (2021) 125GB source code leaked   Cause: No encryption on internal repos	 Facebook (2021) 533 MILLION phone numbers exposed   Cause: Old unencrypted backup	 LinkedIn (2021) 700 MILLION user records scraped   Cause: API not rate-limited
--	--	---



Most breaches are from MISCONFIGURATIONS, not sophisticated hacks!





KEY TAKEAWAY: Simple security controls (encryption, access controls, monitoring) prevent most data breaches.



2.4 Data Security Mechanisms

Application Examples

Data Type	Protection Method	Real-World Service
Aadhaar data	Encrypted at rest (SSE-KMS) + in transit (TLS)	UIDAI servers
Payment info	PCI-DSS compliance requires encryption everywhere	Razorpay, PayTM
Medical records	HIPAA requires encryption + access logging	Practo, 1mg
Source code	Encrypted S3 + IAM restrictions	GitHub private repos
Chat messages	End-to-end encryption	WhatsApp, Signal
Streaming content	DRM + encrypted delivery	Netflix, Hotstar

2.4 Data Security Mechanisms

Data States and Protection

State	Description	Protection
At Rest	Stored in S3, EBS, RDS	AES-256 encryption
In Transit	Network transfer	TLS 1.2+
In Use	Processing in memory	Nitro Enclaves

S3 Encryption Options

Type	Key Management	Use Case
SSE-S3	AWS managed	Default, simple
SSE-KMS	Customer managed via KMS	Audit trail, key rotation
SSE-C	Customer provided	Full control
Client-side	Encrypt before upload	Maximum security

2.4 Data Security Mechanisms

Enable S3 Encryption

```
# Enable default encryption on bucket
aws s3api put-bucket-encryption \
  --bucket my-bucket \
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "aws:kms",
        "KMSEMasterKeyID": "alias/my-key"
      }
    }]
  }'

# Block public access
aws s3api put-public-access-block \
  --bucket my-bucket \
  --public-access-block-configuration '{
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }'
```

2.4 Data Security Mechanisms

Data Loss Prevention (DLP)

DLP scans for sensitive patterns (API keys, SSNs, credit cards) and blocks them from being emailed, uploaded, or pushed to GitHub. Like .gitignore for your entire organization

Amazon Macie - ML-powered sensitive data discovery

```
# Enable Macie
aws macie2 enable-macie

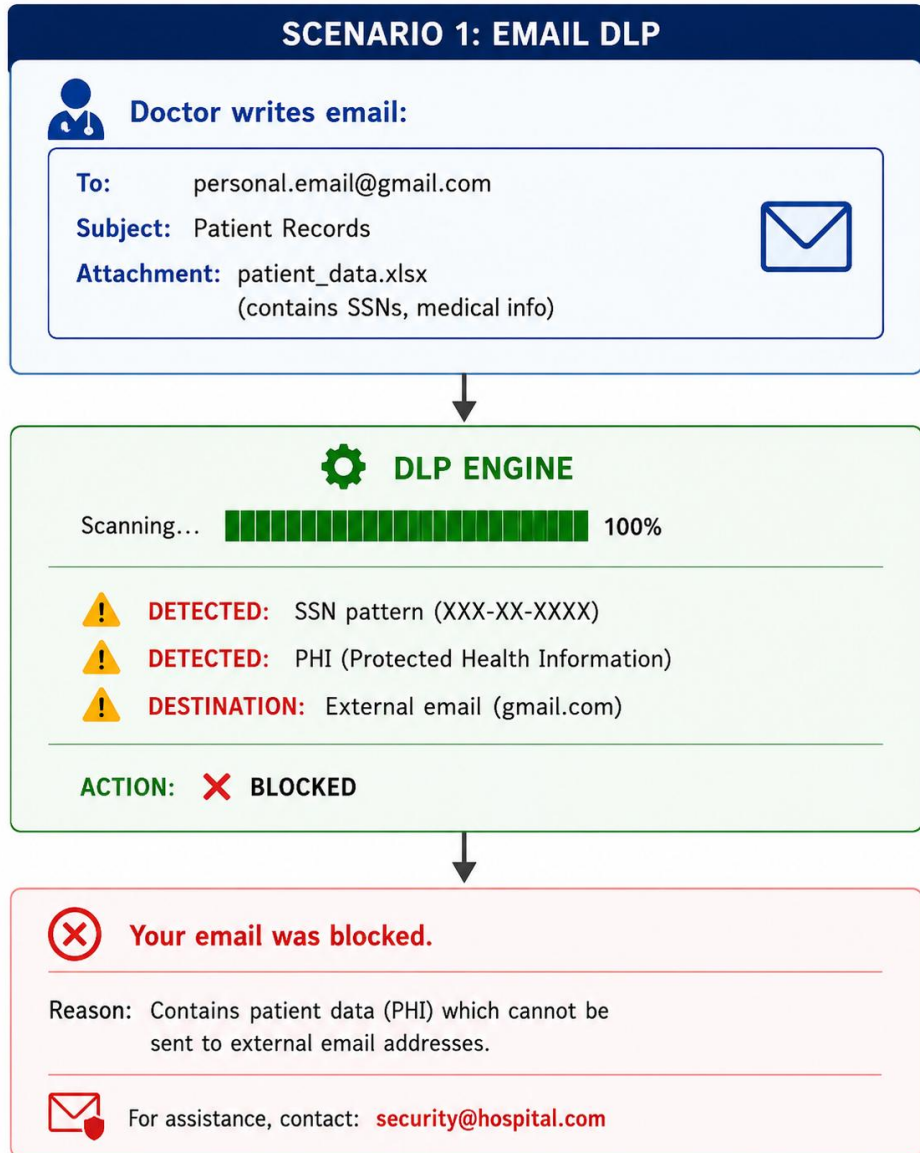
# Create classification job
aws macie2 create-classification-job \
  --job-type ONE_TIME \
  --s3-job-definition '{
    "bucketDefinitions": [{
      "accountId": "123456789012",
      "buckets": ["my-sensitive-bucket"]
    }]
  }' \
  --name "PII-Scan"
```

Detected patterns:

- Credit card numbers
- Social Security Numbers
- API keys and secrets
- Personal health information (PHI)

2.5 Security Monitoring & Governance

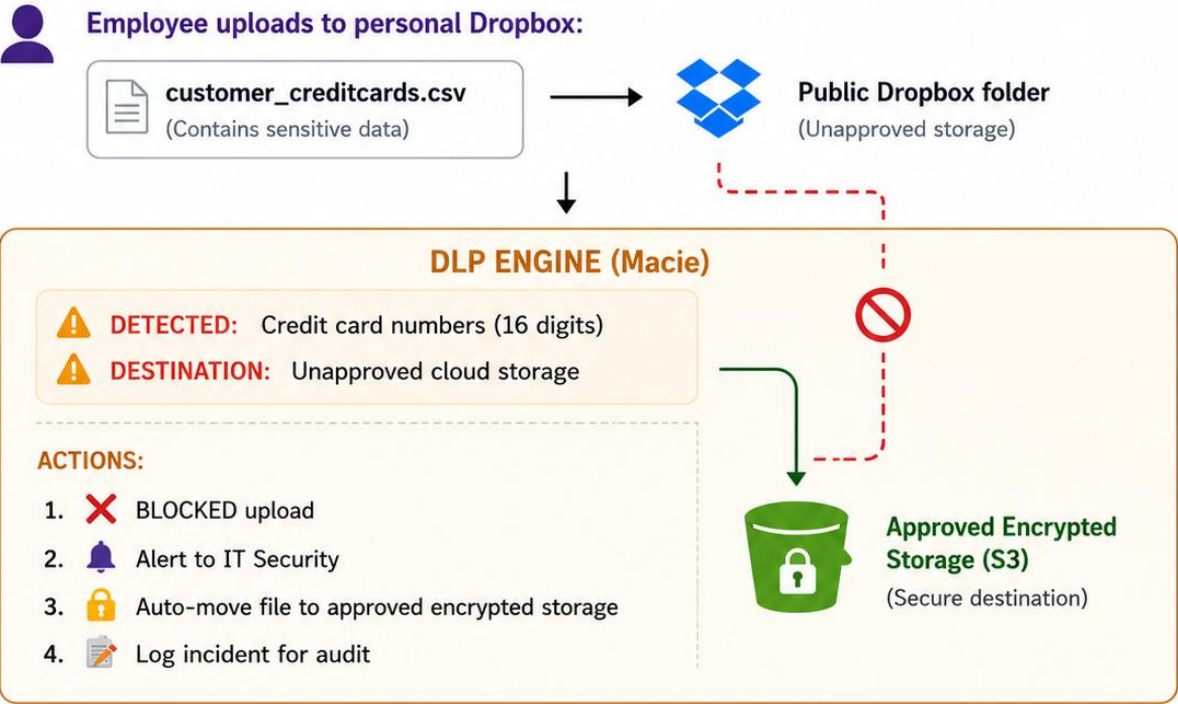
DLP in Action



2.4 Data Security Mechanisms

DLP in Action - Real Scenarios

 **SCENARIO:** Employee uploads sensitive file to public cloud



Macie Scan Results Example:

S3 Bucket: "company-data"

File / Folder	Scan Result	Details
 customers.csv		Contains 1,234 SSNs
 payments.xlsx		Contains 567 Credit Card numbers
 employees.json		Contains PII (names, addresses)
 images/		No sensitive data detected
 logs/		No sensitive data detected

2.4 Data Security Mechanisms



AWS DLP Service - Amazon Macie



AWS MACIE - DLP FOR S3



Amazon Macie automatically scans **S3 buckets** for **sensitive data**.



S3 Bucket: “company-data”

Macie Scan Results:

	File / Folder	Scan Result	Details
	customers.csv	SENSITIVE	Contains 1,234 SSNs
	payments.xlsx	SENSITIVE	Contains 567 Credit Card numbers
	employees.json	SENSITIVE	Contains PII (names, addresses)
	images/	CLEAN	No sensitive data detected
	logs/	CLEAN	No sensitive data detected



Recommendations:



Enable encryption on sensitive files



Move PII to private bucket



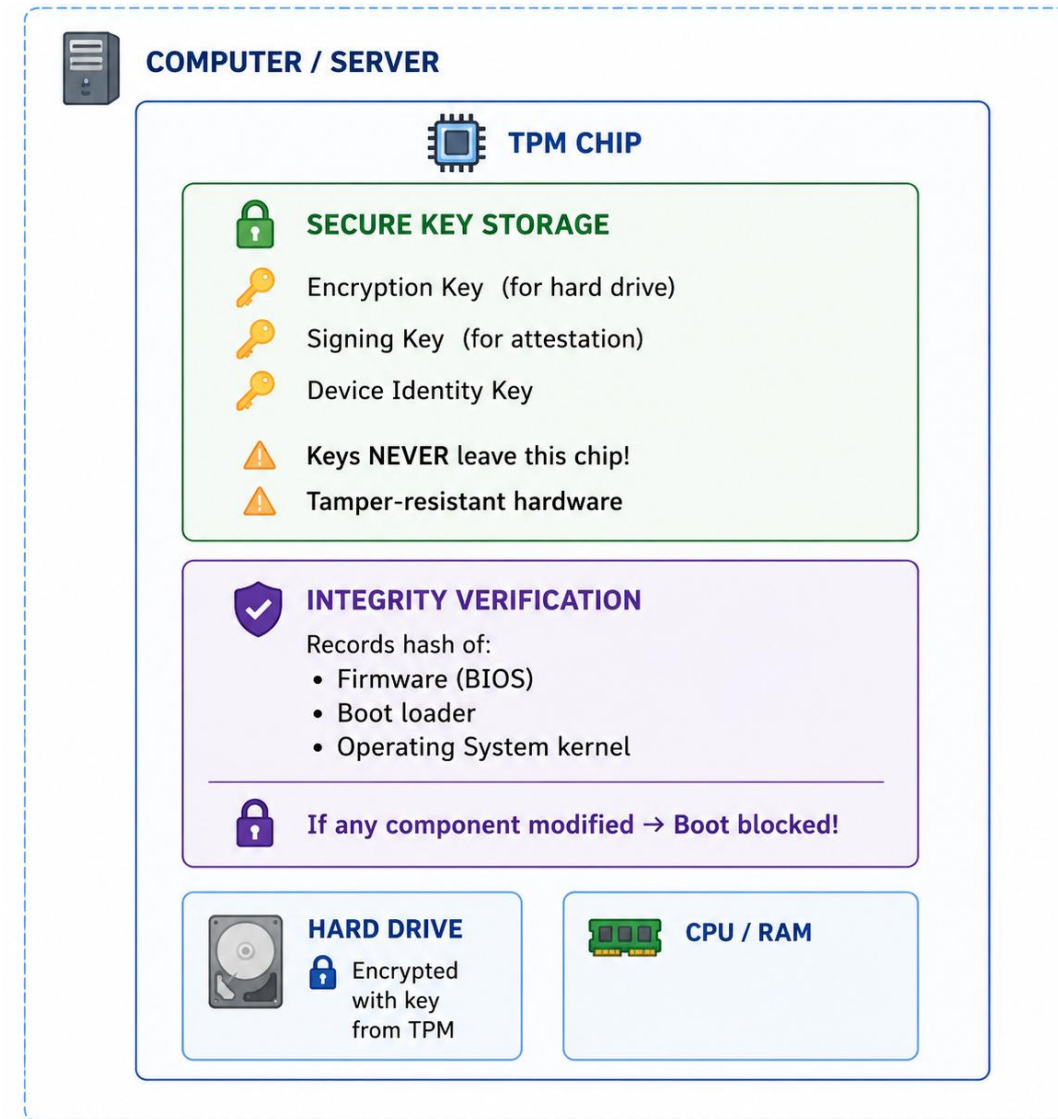
Review access policies

2.4 Data Security Mechanisms

TOPIC B: Trusted Platform Module (TPM)

Core Concept:

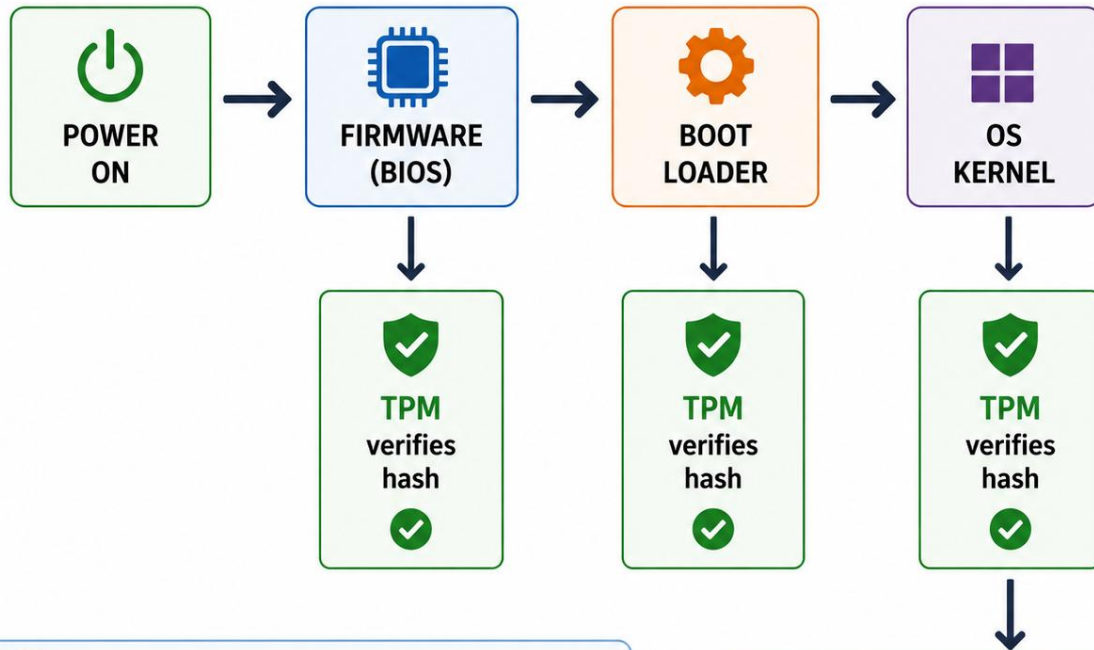
Hardware security chip storing encryption keys. Verifies boot integrity. Even if hard drive is stolen, data is unreadable without TPM chip.



2.4 Data Security Mechanisms

TOPIC B: Trusted Platform Module (TPM)

NORMAL BOOT (Everything verified)



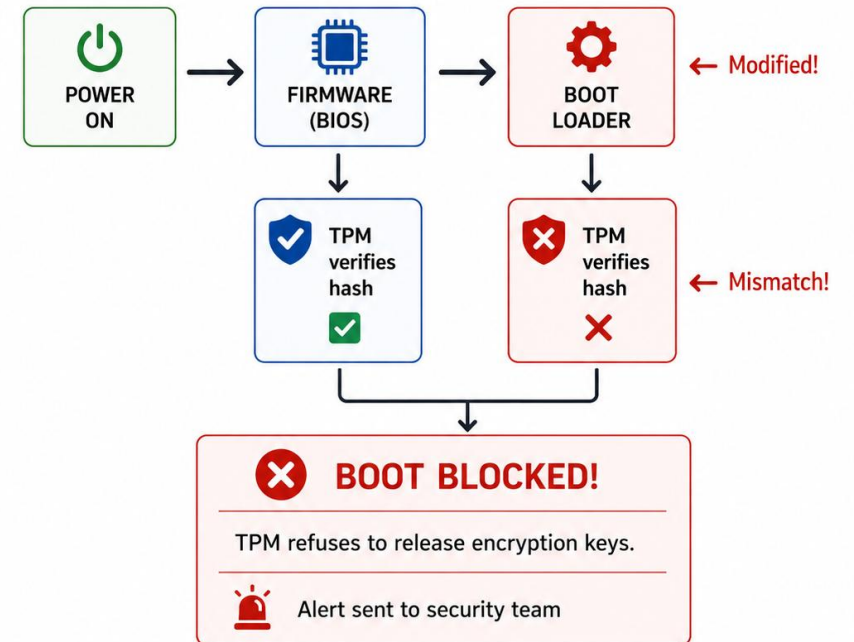
What happens?

- ✓ TPM (Trusted Platform Module) verified the hash of each critical component.
- ✓ All hashes matched the expected values.
- ✓ TPM releases encryption keys.
- ✓ System boots normally and securely.

 **TPM releases encryption keys**
→ **System boots!**

TAMPERED BOOT (Attack detected)

 **Attacker modifies boot loader to install malware...**

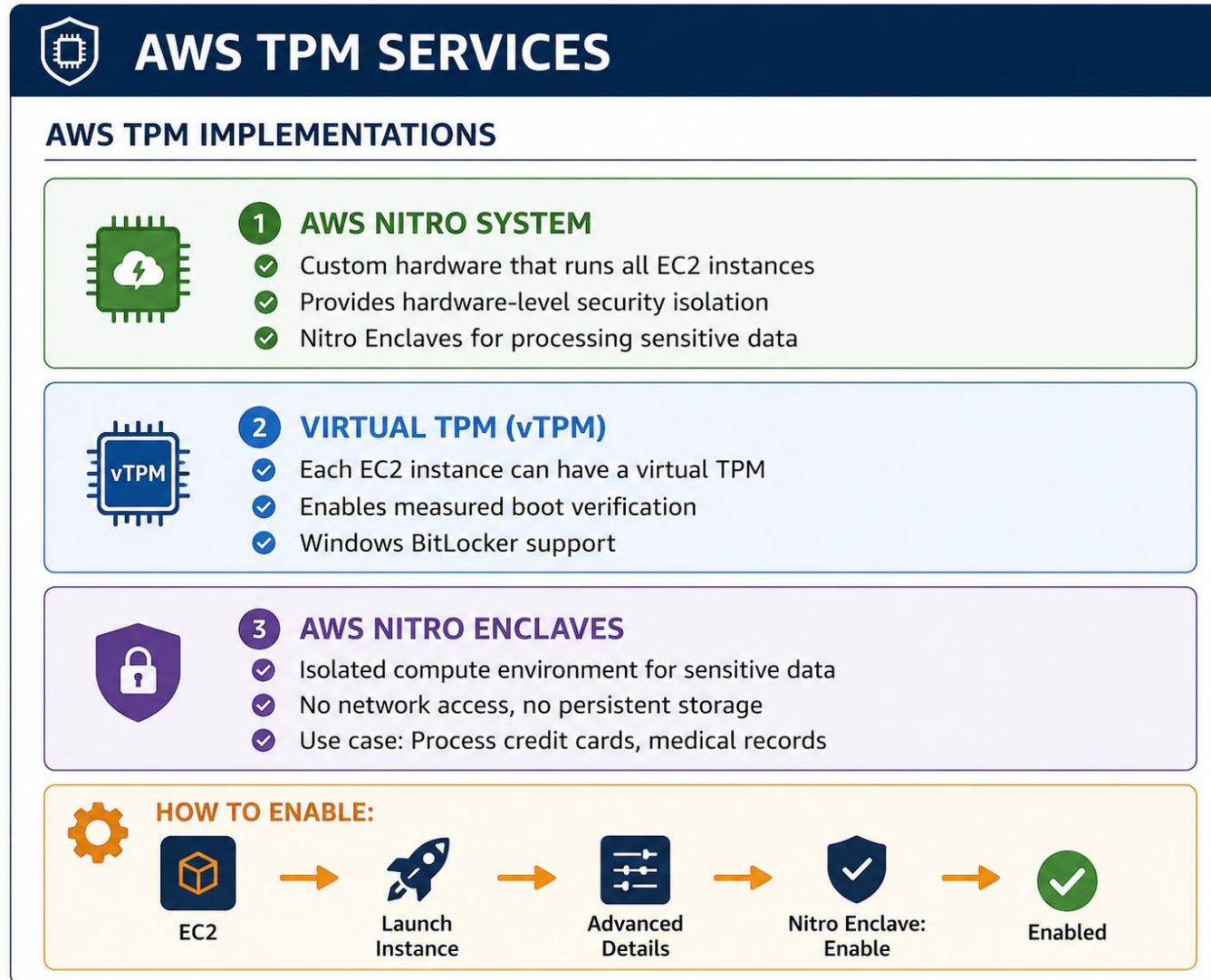


What happened?

- TPM (Trusted Platform Module) verified the hash of each critical component.
- The boot loader hash did not match the expected value.
- System boot was blocked to prevent malware execution.
- No encryption keys were released.

2.4 Data Security Mechanisms

TOPIC B: Trusted Platform Module (TPM)



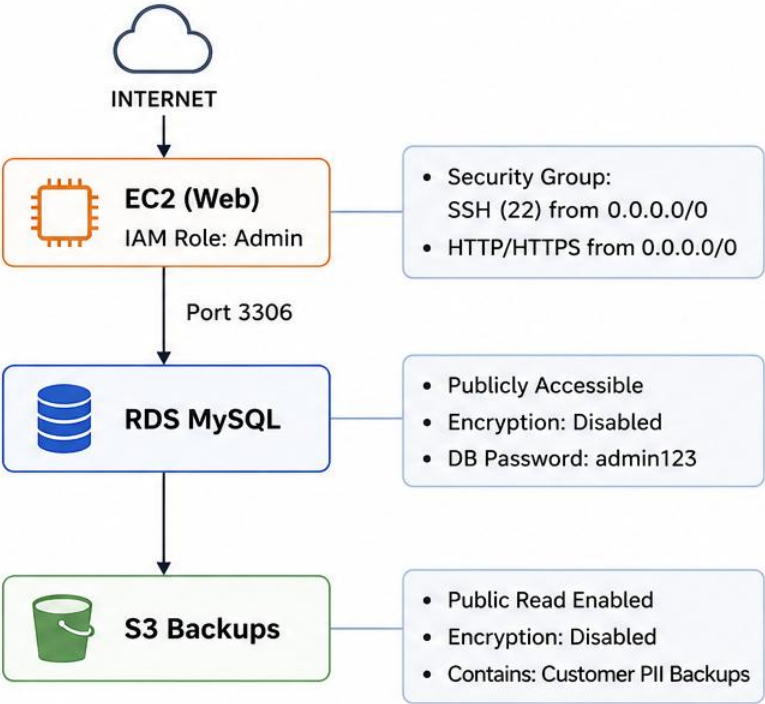
2.4 Data Security Mechanisms



SECURITY REVIEW CHALLENGE

What's Wrong With This AWS Architecture?

QUESTION: Identify as many security issues as possible in this environment.



CONFIGURATION DETAILS

Resource	Configuration
EC2 Security Group	SSH (22) from 0.0.0.0/0
EC2 IAM Role	Administrator Access
Web Access	HTTP/HTTPS from Internet (0.0.0.0/0)
RDS Database	Publicly Accessible
RDS Encryption	Disabled
Database Password	admin123
S3 Bucket	Public Read Enabled
S3 Encryption	Disabled
Data Stored	Customer PII Backups
Logging	Not Configured



YOUR TASK:

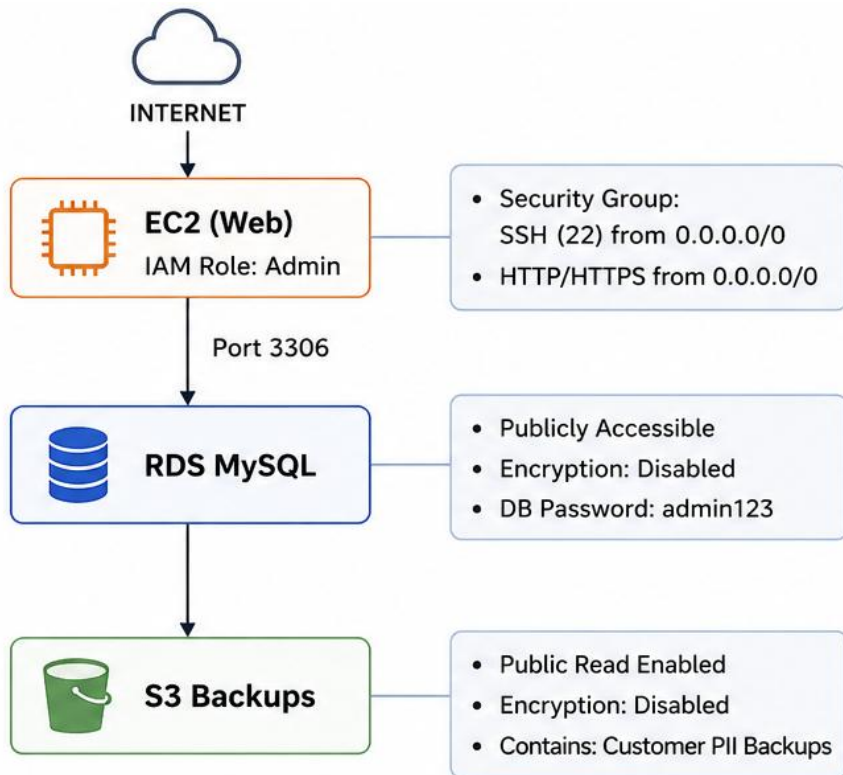
Find at least 8 security issues in this architecture.



DISCUSSION QUESTION:

If an attacker compromises the EC2 instance, what other resources can they access next?

2.4 Data Security Mechanisms



Issue	Risk	Fix
SSH from 0.0.0.0/0	Brute force attacks	Allow SSH only from VPN/bastion
IAM Admin role on EC2	Full account compromise if hacked	Least privilege - only needed permissions
MySQL port 3306 public	Direct database attacks	Allow only from web-sg security group
RDS in public subnet	Exposed to internet	Move to private subnet
RDS no encryption	Data readable if disk stolen	Enable encryption at rest
Weak DB password	Easy to crack	Use Secrets Manager, 20+ char password
S3 public read	Anyone can download backups	Block public access, use IAM policies
S3 no encryption	Data exposed if breached	Enable SSE-KMS encryption
PII in S3	Compliance violation (GDPR, etc.)	Classify data, enable Macie scanning
No CloudTrail	Can't detect attacks	Enable multi-region CloudTrail

This is exactly how Capital One was breached!

2.5 Security Monitoring & Governance



CAMPUS SECURITY ANALOGY



**CloudTrail =
CCTV Footage Recording**

- ✓ Records WHO entered WHICH building at WHAT time
- ✓ "Student A swiped into Lab at 2:30 AM" (API call logged)
- ✓ Can review footage later to investigate incidents
- ✓ Doesn't STOP bad actions, just RECORDS them



**GuardDuty =
Smart AI Security Guard**

- ✓ Watches all CCTV feeds in real-time
- ✓ Notices: "Someone's trying 100 doors at 3 AM" (brute force)
- ✓ Alerts: "Unknown person following students" (suspicious IP)
- ✓ Uses ML to detect unusual patterns you'd miss



**AWS Config =
Building Code Inspector**

- ✓ Checks: "Is fire exit blocked?" (is S3 bucket public?)
- ✓ Reports: "Room 203 violates safety code" (non-compliant)
- ✓ Continuous compliance monitoring



DETECT
(CloudTrail)

+



RECORD
(GuardDuty)

+



PREVENT
(AWS Config)

=



SECURE CAMPUS
(Together)

2.5 Security Monitoring & Governance

CloudTrail - Audit Logging

Records ALL API calls in your AWS account. Think of it as CCTV for your cloud.

```
# Create trail
aws cloudtrail create-trail \
  --name my-trail \
  --s3-bucket-name my-logs-bucket \
  --is-multi-region-trail

# Start Logging
aws cloudtrail start-logging --name my-trail

# Query events
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue>DeleteBucket
```

What gets logged:

Action	CloudTrail Records
Someone creates EC2	Who, when, what size, which region
Someone deletes S3 bucket	Who, when, which bucket, from where (IP)
Failed login attempt	Who tried, when, why it failed
IAM policy change	Who changed what, before/after values

2.5 Security Monitoring & Governance

Guard Duty - Threat Detection

AI-powered threat detection that analyses:

- CloudTrail events (API calls)
- VPC Flow Logs (network traffic)
- DNS logs (domain lookups)
- EKS audit logs (Kubernetes activity)

```
# Enable GuardDuty (one-click security!)
aws guardduty create-detector --enable

# List findings
aws guardduty list-findings --detector-id abc123

# Get finding details
aws guardduty get-findings --detector-id abc123 --finding-ids finding-id-1
```

Real-World Finding Examples:

Finding	What It Means	Real Scenario
UnauthorizedAccess:EC2/TorClient	EC2 talking to Tor network	Attacker hiding tracks
CryptoCurrency:EC2/BitcoinTool	Crypto mining detected	Your bill about to explode!
Recon:EC2/PortProbeUnprotectedPort	Someone scanning your ports	Attacker mapping your infra
Trojan:EC2/DNSDataExfiltration	Data being stolen via DNS	Capital One style attack
Impact:EC2/WinRMBruteForce	Windows password guessing	Bot trying to break in

2.5 Security Monitoring & Governance

AWS Config - Compliance

Continuous compliance monitoring with rules. Like having an automated auditor.

```
# Enable Config
aws configservice put-configuration-recorder \
  --configuration-recorder name=default,roleARN=arn:aws:iam::123456789:role/ConfigRole

# Add managed rule
aws configservice put-config-rule \
  --config-rule '{
    "ConfigRuleName": "s3-bucket-public-read-prohibited",
    "Source": {
      "Owner": "AWS",
      "SourceIdentifier": "S3_BUCKET_PUBLIC_READ_PROHIBITED"
    }
  }'
```

Common rules:

Rule	What It Checks	Why It Matters
s3-bucket-public-read-prohibited	Is any S3 bucket public?	Prevents data leaks
ec2-instance-no-public-ip	Does EC2 have public IP?	Forces traffic through LB
iam-password-policy	Is password policy strong?	Prevents weak passwords
rds-storage-encrypted	Is database encrypted?	Compliance requirement
vpc-flow-logs-enabled	Are network logs on?	Required for forensics

2.6 Lab Exercise: Secure AWS Environment

Lab Overview

Task	What You'll Do
1. Secure Root Account	Enable MFA, create admin user
2. Create VPC Architecture	Public + private subnets
3. Configure Security Groups	Web, app, and DB tiers
4. Enable Monitoring	CloudTrail, GuardDuty, Config
5. Set Up Billing Alert	\$10 threshold notification
6. Verify & Cleanup	Test access, terminate resources

Objective

Implement security best practices on an AWS account.

2.6 Lab Exercise: Secure AWS Environment

Task 1: Secure Root Account

In AWS Console:

1. Click your account name (top right) → **Security credentials**
2. Under **Multi-factor authentication (MFA)** → **Assign MFA device**
3. Choose **Authenticator app** → scan QR with Google Authenticator
4. Enter two consecutive codes → **Add MFA**

Then create admin user:

```
# Or via CLI after console setup:  
aws iam create-user --user-name admin  
aws iam attach-user-policy \  
    --user-name admin \  
    --policy-arn arn:aws:iam::aws:policy/AdministratorAccess  
# Enable MFA via console for the admin user too!
```

 **IMPORTANT:** After this, STOP using root account. Use your new admin user!

2.6 Lab Exercise: Secure AWS Environment

Task 2: Create Secure VPC Architecture

In AWS Console: VPC → Create VPC

Setting	Value
Name	secure-vpc
IPv4 CIDR	10.0.0.0/16
Tenancy	Default

Create Subnets:

Subnet	CIDR	AZ	Purpose
public-subnet	10.0.1.0/24	us-east-1a	Web servers
private-subnet	10.0.2.0/24	us-east-1a	Databases

Via CLI:

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16 --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=secure-vpc}]'
```

Note the VPC ID from output, then:

```
aws ec2 create-subnet --vpc-id vpc-xxx --cidr-block 10.0.1.0/24 --availability-zone us-east-1a
```

```
aws ec2 create-subnet --vpc-id vpc-xxx --cidr-block 10.0.2.0/24 --availability-zone us-east-1a
```

2.6 Lab Exercise: Secure AWS Environment

Task 3: Configure Security Groups

Create three security groups:

Security Group	Inbound Rules	Purpose
web-sg	80, 443 from 0.0.0.0/0; 22 from YOUR IP only	Web servers
app-sg	8080 from web-sg only	Application tier
db-sg	3306 from app-sg only	Database tier

In Console: EC2 → Security Groups → Create security group

```
# Create web security group
aws ec2 create-security-group \
  --group-name web-sg \
  --description "Web tier - public access" \
  --vpc-id vpc-xxx

# Allow HTTP/HTTPS from anywhere
aws ec2 authorize-security-group-ingress \
  --group-id sg-web \
  --protocol tcp --port 80 --cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress \
  --group-id sg-web \
  --protocol tcp --port 443 --cidr 0.0.0.0/0

# Allow SSH only from your IP (replace with your actual IP)
aws ec2 authorize-security-group-ingress \
  --group-id sg-web \
  --protocol tcp --port 22 --cidr $(curl -s ifconfig.me)/32
```

2.6 Lab Exercise: Secure AWS Environment

Task 4: Enable Monitoring

Enable CloudTrail:

1. Search "CloudTrail" → **Create trail**
2. Name: `security-audit-trail`
3. Apply to all regions: **Yes**
4. Create new S3 bucket for logs

Enable GuardDuty:

1. Search "GuardDuty" → **Get Started** → **Enable GuardDuty**
2. That's it! GuardDuty starts analyzing immediately.

Via CLI:

```
aws cloudtrail create-trail \  
    --name security-audit-trail \  
    --s3-bucket-name my-cloudtrail-logs-$(date +%s) \  
    --is-multi-region-trail  
  
aws cloudtrail start-logging --name security-audit-trail  
  
aws guardduty create-detector --enable
```

2.6 Lab Exercise: Secure AWS Environment

Task 5: Set Up Billing Alert

Critical for students! Avoid surprise bills.

1. Search "Billing" → **Budgets** → **Create budget**
2. Choose **Cost budget**
3. Budget amount:
4. Email: your email address

```
# Create billing alarm (requires us-east-1)
aws cloudwatch put-metric-alarm \
  --alarm-name "BillingAlert-10USD" \
  --metric-name EstimatedCharges \
  --namespace AWS/Billing \
  --statistic Maximum \
  --period 86400 \
  --threshold 10 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:us-east-1:YOUR_ACCOUNT:billing-alerts \
  --dimensions Name=Currency,Value=USD
```

2.6 Lab Exercise: Secure AWS Environment

Task 6: Verify & Cleanup

Verification Checklist:

- ☐ Can you login with admin user (not root)?
- ☐ Is MFA enabled on both root and admin?
- ☐ Do Security Groups show correct rules?
- ☐ Is CloudTrail logging events?
- ☐ Is GuardDuty enabled and showing status?

Cleanup (to avoid charges):

```
# Delete resources you created:  
aws ec2 delete-security-group --group-id sg-xxx  
aws ec2 delete-subnet --subnet-id subnet-xxx  
aws ec2 delete-vpc --vpc-id vpc-xxx  
# Keep CloudTrail and GuardDuty for Learning (minimal cost)
```

Security Checklist



IDENTITY

- ✓ Enable MFA on root account
- ✓ Enable MFA on all IAM users
- ✓ Use roles instead of access keys
- ✓ Apply least privilege (no AdministratorAccess!)
- ✓ Review IAM access quarterly



NETWORK

- ✓ No SSH (22) from 0.0.0.0/0
- ✓ No RDP (3389) from 0.0.0.0/0
- ✓ Database in private subnet only
- ✓ Use security group references (not IPs)



DATA

- ✓ Enable S3 default encryption
- ✓ Block S3 public access
- ✓ Enable RDS encryption
- ✓ Use TLS 1.2+ for all connections



MONITORING

- ✓ Enable CloudTrail (all regions)
- ✓ Enable GuardDuty
- ✓ Set billing alert (\$10 threshold)
- ✓ Enable VPC Flow Logs



TIP: Review this checklist regularly and automate where possible!

Architecture Pattern

Pattern / Architecture	Use When...
Workload Distribution	High traffic needs reliability
Elastic Capacity	Variable / unpredictable traffic
Multi-Cloud	Need redundancy; avoid vendor lock-in
Hypervisor Cluster	Need high availability / live migration
Cloud Balancing	Global users need low latency everywhere
Edge Computing	Real-time decisions; processing close to users
Fog Computing	IoT with regional processing capability
Metacloud	Managing multiple clouds from one place
Federated Cloud	Organizations sharing resources while staying independent

Cloud Resources

Pattern / Architecture	Cloud Solution
Workload Distribution	Use Load Balancers (ELB)
Elastic Capacity	Use Auto Scaling Groups
Multi-Cloud	Avoid vendor lock-in
Hypervisor Cluster	High Availability
Cloud Balancing	Route 53 for smart routing
Edge Computing	Process near the data (CloudFront)
Fog Computing	Middle layer (IoT Greengrass)
Metacloud	Kubernetes for multi-cloud
Federated Cloud	Independent clouds cooperating

AWS Services at Glance

AWS Service	One-Line Description
EC2	Virtual servers in the cloud
S3	Object storage for files, images, and backups
RDS	Managed relational databases like MySQL & PostgreSQL
VPC	Private networking environment in AWS
IAM	Identity and access management for users and permissions
Security Groups	Virtual firewalls controlling inbound/outbound traffic
Route 53	DNS management and traffic routing service
CloudFront	Content Delivery Network (CDN) for faster access

AWS Services at Glance

AWS Service	One-Line Description
ELB	Load Balancer to distribute application traffic
Auto Scaling	Automatically scale servers based on demand
CloudTrail	Monitor and log AWS account activities
GuardDuty	Intelligent threat detection and security monitoring
Macie	Detect and protect sensitive data in S3
KMS	Encryption key creation and management service
EKS	Managed Kubernetes service on AWS
IoT Greengrass	Run AWS services on edge and fog devices

Certification Path

1. **Cloud Practitioner** (foundational) - 20 hrs study
2. **Solutions Architect Associate** (most valuable) - 60 hrs study
3. **Security Specialty** or **DevOps Professional** - 80+ hrs study



THANK YOU!

Good luck on your cloud journey